

Documento Técnico: Despliegue Independiente de la Aplicación — Proyecto HeartCoin

Introducción

Este documento describe el proceso mediante el cual la aplicación cliente de HeartCoin se prepara, firma y empaqueta para su distribución fuera del entorno de desarrollo, sin depender de un servicio de integración o despliegue continuo. Cubre la configuración de identidad de la aplicación, el mecanismo de firma criptográfica de los paquetes de release, la gestión de la configuración de backend embebida en el cliente, y el estado real de avance hacia la publicación en Google Play Store. Se documenta el proceso efectivamente ejecutado sobre este proyecto, no un procedimiento genérico de referencia.

Objetivo

Documentar el mecanismo de compilación y firma independiente del paquete de distribución de HeartCoin para Android, la configuración de identidad de la aplicación ante la tienda de destino, y el estado de avance real del proceso de publicación, de modo que cualquier persona del equipo pueda retomar o repetir el proceso sin depender de conocimiento no documentado.

Alcance

Este documento cubre el despliegue independiente de la aplicación para la plataforma Android, incluyendo la configuración de `applicationId`, la generación y uso de la clave de firma de release, la generación del paquete `.aab`, y los pasos manuales pendientes en Google Play Console. No cubre la plataforma iOS: el proyecto contiene el esqueleto de un proyecto Xcode (`ios/Runner.xcodeproj`), pero no se ha ejecutado ningún paso de preparación de release para esa plataforma (sin certificados, sin provisioning profiles, sin `Podfile` generado), por lo que no hay proceso real que documentar en ese frente. Tampoco cubre infraestructura de integración continua, por no existir ningún pipeline configurado en el repositorio (se confirmó la ausencia de directorios o archivos de configuración de GitHub Actions, GitLab CI, Bitrise o Codemagic).

Metodología

La información se obtuvo mediante inspección directa de los archivos de configuración de compilación del proyecto (`android/app/build.gradle.kts`, `android/key.properties`, `pubspec.yaml`, `android/gradle/wrapper/gradle-wrapper.properties`), y se contrastó contra el estado real de avance

descrito en la documentación interna del proyecto y contra la salida efectiva del proceso de compilación ya ejecutado sobre este repositorio.

Desarrollo del Análisis

1. Identidad de la Aplicación

La aplicación se distribuye bajo un identificador de paquete único y permanente ante la tienda de destino. Este identificador se declara en dos puntos del mismo archivo de configuración de Gradle, y ambos deben coincidir:

```
// android/app/build.gradle.kts
android {
    namespace = "com.theoriginallab.heartcoin"
    ...
    defaultConfig {
        applicationId = "com.theoriginallab.heartcoin"
        ...
    }
}
```

Este valor reemplazó al identificador de plantilla generado automáticamente por Flutter al crear el proyecto (`com.example.heartcoin`), el cual no es apto para publicación por corresponder a un dominio de ejemplo reservado. La decisión de adoptar `com.theoriginallab.heartcoin` es **irreversible una vez publicada la primera versión**: Google Play asocia la ficha de la aplicación de forma permanente a este identificador, y cambiarlo posteriormente equivale a crear una aplicación nueva sin historial, sin reseñas y sin la base de usuarios existente.

El resto de los parámetros de identidad de la aplicación — `compileSdk`, `minSdk`, `targetSdk`, `versionCode` y `versionName` — no se fijan directamente en este archivo, sino que se heredan de los valores que el propio SDK de Flutter determina como recomendados (`flutter.compileSdkVersion`, `flutter.minSdkVersion`, etc.), y del campo `version:` declarado en `pubspec.yaml` (actualmente `1.0.0+1`, equivalente a `versionName 1.0.0` y `versionCode 1`).

2. Firma Criptográfica del Paquete de Release

Todo paquete de Android distribuido fuera del entorno de desarrollo debe estar firmado con una clave criptográfica que identifica de forma única al autor de la aplicación ante la tienda de destino. Durante el desarrollo inicial del proyecto, los paquetes de prueba se firmaban automáticamente con la clave de depuración (*debug key*) que Flutter genera por proyecto — una clave sin valor de identidad real, que Google Play rechaza de forma explícita para cualquier publicación.

Se generó una clave de firma de release real mediante la herramienta `keytool` (incluida en el JDK), con una validez configurada a largo plazo. Esta clave se referencia desde la configuración de Gradle a través de un archivo intermedio de propiedades, para evitar que las credenciales de la clave queden escritas directamente en el archivo de build (que sí se versiona en el control de código fuente):

```
// android/app/build.gradle.kts
val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

signingConfigs {
    create("release") {
        if (keystorePropertiesFile.exists()) {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword = keystoreProperties["keyPassword"] as String
            storeFile = file(keystoreProperties["storeFile"] as String)
            storePassword = keystoreProperties["storePassword"] as String
        }
    }
}

buildTypes {
    release {
        signingConfig = signingConfigs.getByName("release")
    }
}
```

El archivo `android/key.properties` contiene las cuatro propiedades referenciadas (`storePassword`, `keyPassword`, `keyAlias`, `storeFile`), y el archivo binario de la clave en sí reside en `android/keystore/heartcoin-upload.jks`. Ambos archivos están deliberadamente excluidos del control de versiones mediante `.gitignore`, por tratarse de material criptográfico sensible cuya exposición pública comprometería la identidad de firma de la aplicación de forma irreversible.

Riesgo operativo crítico: la clave de firma de release no tiene ningún mecanismo de respaldo automatizado dentro del proyecto, por diseño (no debe versionarse). Si el archivo `heartcoin-upload.jks` se pierde o corrompe sin una copia de seguridad externa, no será posible publicar ninguna actualización futura sobre la misma ficha de la aplicación en Google Play sin iniciar un proceso de recuperación de cuenta directamente con Google, cuyo resultado no está

garantizado. Este archivo debe respaldarse en un almacenamiento seguro fuera del repositorio antes de continuar con cualquier ciclo de publicación adicional.

3. Compatibilidad de Herramientas de Compilación

La generación del paquete de release depende de una cadena de herramientas de compilación (Gradle, el plugin de Android y Kotlin) cuyas versiones están fijadas explícitamente en el proyecto: Gradle 8.14 (declarado en `android/gradle/wrapper/gradle-wrapper.properties`), Android Gradle Plugin 8.11.1 y Kotlin 2.2.20 (ambos en `android/settings.gradle.kts`). Estas versiones producen advertencias de compatibilidad hacia adelante durante la compilación (Flutter anuncia que dejará de soportarlas próximamente), pero no impiden la generación exitosa del paquete al día de este documento.

Adicionalmente, la compilación requiere *core library desugaring* habilitado explícitamente:

```
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
    isCoreLibraryDesugaringEnabled = true
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}
```

Este requisito no es opcional: sin él, la compilación falla explícitamente al incluir la dependencia `flutter_local_notifications`, cuya implementación nativa en Android requiere APIs de Java más recientes que las disponibles de forma nativa en versiones antiguas de Android, resueltas mediante esta capa de compatibilidad (*desugaring*).

4. Configuración de Backend Embebida en el Cliente

A diferencia de un esquema de despliegue con separación de ambientes (desarrollo, staging, producción) mediante variables de entorno o parámetros de compilación (`--dart-define`), la aplicación HeartCoin embebe directamente en el código fuente la URL del proyecto de Supabase y la clave pública (*anon key*) utilizadas en tiempo de ejecución:

```
// lib/main.dart
await Supabase.initialize(
    url: 'https://bamrvwzpcqwoyuwbvomw.supabase.co',
```

```
anonKey: 'sb_publishable_h_Xb1Hwas6LzD4wWK-nEUQ_kYI3qs2m',  
);
```

Esta clave es de tipo público (*publishable/anon key*), diseñada explícitamente por Supabase para ser expuesta en clientes distribuidos —su seguridad depende de las políticas de Row Level Security configuradas en la base de datos, no de mantener la clave en secreto—, por lo que su presencia en el código fuente no constituye, por sí misma, una vulnerabilidad. Sin embargo, esta configuración implica que **todo paquete compilado, sin importar el canal de distribución, apunta siempre al mismo proyecto de base de datos**: no existe hoy una forma de generar un build de prueba contra un proyecto de Supabase distinto sin editar manualmente este archivo antes de compilar.

5. Generación del Paquete de Distribución (Android App Bundle)

Google Play exige, desde 2021, que las aplicaciones nuevas se publiquen en formato *Android App Bundle* (`.aab`) en lugar del formato histórico `.apk` . Este formato delega en la propia infraestructura de Google Play la generación de los paquetes de instalación optimizados por configuración de dispositivo (arquitectura de procesador, densidad de pantalla, idioma), en lugar de distribuir un paquete único con todos los recursos posibles incluidos.

El comando de compilación utilizado es el estándar de Flutter para este formato:

```
flutter build appbundle --release
```

Este comando, ejecutado sobre la configuración descrita en las secciones anteriores, generó exitosamente el archivo `app-release.aab` (72.6 MB), firmado con la clave de release real y verificado como válido para su carga en Play Console. La generación de este archivo no requiere ninguna intervención manual adicional más allá de tener correctamente configurados el `applicationId` y el `signingConfig` descritos previamente.

6. Estado del Proceso de Publicación en Google Play

La preparación técnica del lado del cliente está completa; el resto del proceso de publicación depende de acciones manuales dentro de Google Play Console, sobre una cuenta de desarrollador que ya existe para el proyecto:

Elemento	Estado
Identificador de aplicación (<code>applicationId</code>)	Definido y aplicado (<code>com.theoriginallab.heartcoin</code>)
Clave de firma de release	Generada y en uso; pendiente de respaldo externo

Elemento	Estado
Paquete <code>.aab</code> de release	Generado y verificado (72.6 MB)
Política de privacidad	Publicada en <code>theoriginallab.com</code> ; pendiente marcarla como compartida en Play Console
Carga del <code>.aab</code> a la pista de pruebas internas	Pendiente, manual
Cuestionario de seguridad de los datos	Pendiente, manual — debe declarar recopilación de ubicación
Capturas de pantalla y descripción de la ficha	Pendiente, manual
Declaración de contenido restringido	Pendiente, manual — aplica, dado que la aplicación exige inicio de sesión; se dispone de cuentas de demostración documentadas para el equipo de revisión
Términos y condiciones	Fuera de alcance por ahora — la sección correspondiente dentro de la aplicación indica explícitamente "Próximamente"

Ninguno de los pasos pendientes requiere cambios adicionales de código: son formularios y cargas de archivo dentro de la consola de Google Play, ejecutables por cualquier persona con acceso a la cuenta de desarrollador y a los archivos generados en las secciones anteriores.

Problemas Detectados

- Ausencia de respaldo verificado de la clave de firma de release:** el archivo `heartcoin-upload.jks` existe únicamente en el entorno local donde se generó, sin evidencia de una copia de seguridad en un almacenamiento externo al equipo de desarrollo.
- Configuración de backend única, sin separación de ambientes:** al no existir mecanismo de variables de entorno o `--dart-define`, cualquier compilación del proyecto —de prueba o de producción— apunta al mismo proyecto de Supabase, sin posibilidad de aislar datos de prueba de datos reales sin editar manualmente el código fuente antes de compilar.
- Dependencia de versiones de herramientas próximas a quedar obsoletas:** Gradle 8.14, Android Gradle Plugin 8.11.1 y Kotlin 2.2.20 generan advertencias de discontinuación durante cada compilación; si Flutter retira el soporte antes de actualizar estas versiones, futuras compilaciones podrían fallar sin previo aviso adicional al ya emitido.

4. **Ausencia total de integración continua:** no existe ningún pipeline automatizado que compile, pruebe o valide el proyecto ante cada cambio; todo el proceso de compilación y firma depende de ejecución manual local, sin verificación automática de que el estado del repositorio sea siempre compilable.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto crítico e irreversible**: la pérdida de la clave de firma, sin respaldo, imposibilita publicar actualizaciones sobre la ficha existente en Google Play, obligando a un proceso de recuperación con resultado incierto o, en el peor caso, a publicar la aplicación como una ficha nueva sin historial.
- El riesgo del punto 2 es de **impacto operativo y de seguridad de datos**: una compilación de prueba distribuida por error con la configuración de producción actual escribiría datos de prueba directamente sobre la base de datos real utilizada por usuarios finales.
- El riesgo del punto 3 es de **impacto en continuidad del desarrollo**, materializable solo si las versiones no se actualizan antes de que Flutter retire el soporte anunciado.
- El riesgo del punto 4 es de **impacto en calidad y velocidad de entrega**: cualquier error de compilación se detecta hasta el momento en que alguien ejecuta el build manualmente, no de forma preventiva ante cada cambio subido al repositorio.

Fortalezas Identificadas

- **Separación correcta entre credenciales sensibles y código versionado**: tanto la clave de firma como el archivo de propiedades que la referencia están explícitamente excluidos del control de versiones, evitando su exposición accidental en el historial del repositorio.
- **Migración completa y verificada de clave de depuración a clave de release real**, condición indispensable para la aceptación de la aplicación por parte de Google Play.
- **Generación exitosa y reproducible del paquete de distribución** mediante el comando estándar de Flutter, sin pasos manuales adicionales ni scripts personalizados de empaquetado.
- **Documentación interna ya existente del estado de publicación** (`MANUAL_TECNICO.md`), que permite retomar el proceso sin depender de memoria institucional no escrita.

Áreas de Mejora

- Establecer un procedimiento formal de respaldo de la clave de firma de release en al menos un almacenamiento externo seguro, con verificación periódica de su integridad.

- Evaluar la incorporación de separación de ambientes mediante `--dart-define` o un mecanismo equivalente, de modo que sea posible compilar variantes de prueba sin riesgo de escritura accidental sobre datos de producción.
- Planificar la actualización de Gradle, Android Gradle Plugin y Kotlin a versiones soportadas de forma sostenida, antes de que Flutter retire el soporte a las versiones actuales.
- Evaluar la incorporación de un pipeline mínimo de integración continua que al menos ejecute `flutter analyze` y las pruebas automatizadas existentes ante cada cambio, como primera línea de defensa antes de cualquier compilación de release.

Recomendaciones

1. Respalidar de inmediato el archivo `android/keystore/heartcoin-upload.jks` y el contenido de `android/key.properties` en un gestor de contraseñas o almacenamiento cifrado fuera del equipo de desarrollo, dado que este es el riesgo de mayor impacto potencial identificado en este documento.
2. Completar los pasos manuales pendientes en Google Play Console (pista de pruebas internas, cuestionario de seguridad de datos, ficha de la aplicación) como siguiente paso inmediato, dado que la preparación técnica ya está completa y no depende de trabajo adicional de desarrollo.
3. Incorporar separación de configuración por ambiente en un ciclo de mejora posterior a la primera publicación, priorizando no bloquear el lanzamiento inicial por esta mejora de naturaleza incremental.
4. Postergar la evaluación de integración continua hasta después de la primera publicación exitosa, dado que el volumen actual de cambios no justifica aún la inversión de configurarla, pero debe reconsiderarse si el ritmo de desarrollo se incrementa.

Conclusión

El despliegue independiente de HeartCoin para Android está técnicamente completo: la aplicación cuenta con un identificador de paquete definitivo, una clave de firma de release real y funcional, y un paquete `.aab` generado y verificado, todo ello sin depender de infraestructura de integración continua ni de servicios externos de automatización. El proceso completo se ejecuta de forma manual y reproducible mediante herramientas estándar de Flutter y Android. Lo que resta para completar la publicación son exclusivamente pasos administrativos dentro de Google Play Console, no trabajo de ingeniería adicional. El riesgo más significativo identificado —la ausencia de respaldo externo de la clave de firma— es de naturaleza operativa y completamente mitigable sin cambios al código ni al proceso de compilación, y debe atenderse antes de considerar cerrado el ciclo de despliegue.